

MANUAL TÉCNICO

Plataforma Azaria - Sistema de Adherencia Terapéutica en Rehabilitación Protésica

Versión: 1.0 **Fecha:** Abril 2026 **Autora:** Mariana Hernández Dimas **Empresa:** C2DEVELOPERS **Organización cliente:** UIOP - UNAM ENES Juriquilla

Índice

- 1. Introducción
 - 2. Arquitectura del Sistema
 - 3. Requisitos del Sistema
 - 4. Estructura del Proyecto
 - 5. Base de Datos
 - 6. Backend - API RESTful
 - 7. Frontend - Aplicación React
 - 8. Sistema de Autenticación
 - 9. Sistema de Accesibilidad
 - 10. Progressive Web App (PWA)
 - 11. Catálogo de Rutas API
 - 12. Configuración del Entorno de Desarrollo
 - 13. Configuración del Entorno de Producción
 - 14. Procedimientos de Despliegue
 - 15. Mantenimiento y Administración
 - 16. Solución de Problemas
 - 17. Glosario
-

1. Introducción

1.1 Propósito del documento

Este manual técnico proporciona la documentación completa de la plataforma Azaria, incluyendo su arquitectura, configuración, instalación, operación y mantenimiento. Está dirigido a desarrolladores, administradores de sistemas y personal técnico responsable del mantenimiento y evolución del sistema.

1.2 Alcance

El documento cubre todos los componentes del sistema: el frontend (aplicación React), el backend (API PHP), la base de datos (MySQL), la configuración de la PWA, y los procedimientos de despliegue y mantenimiento.

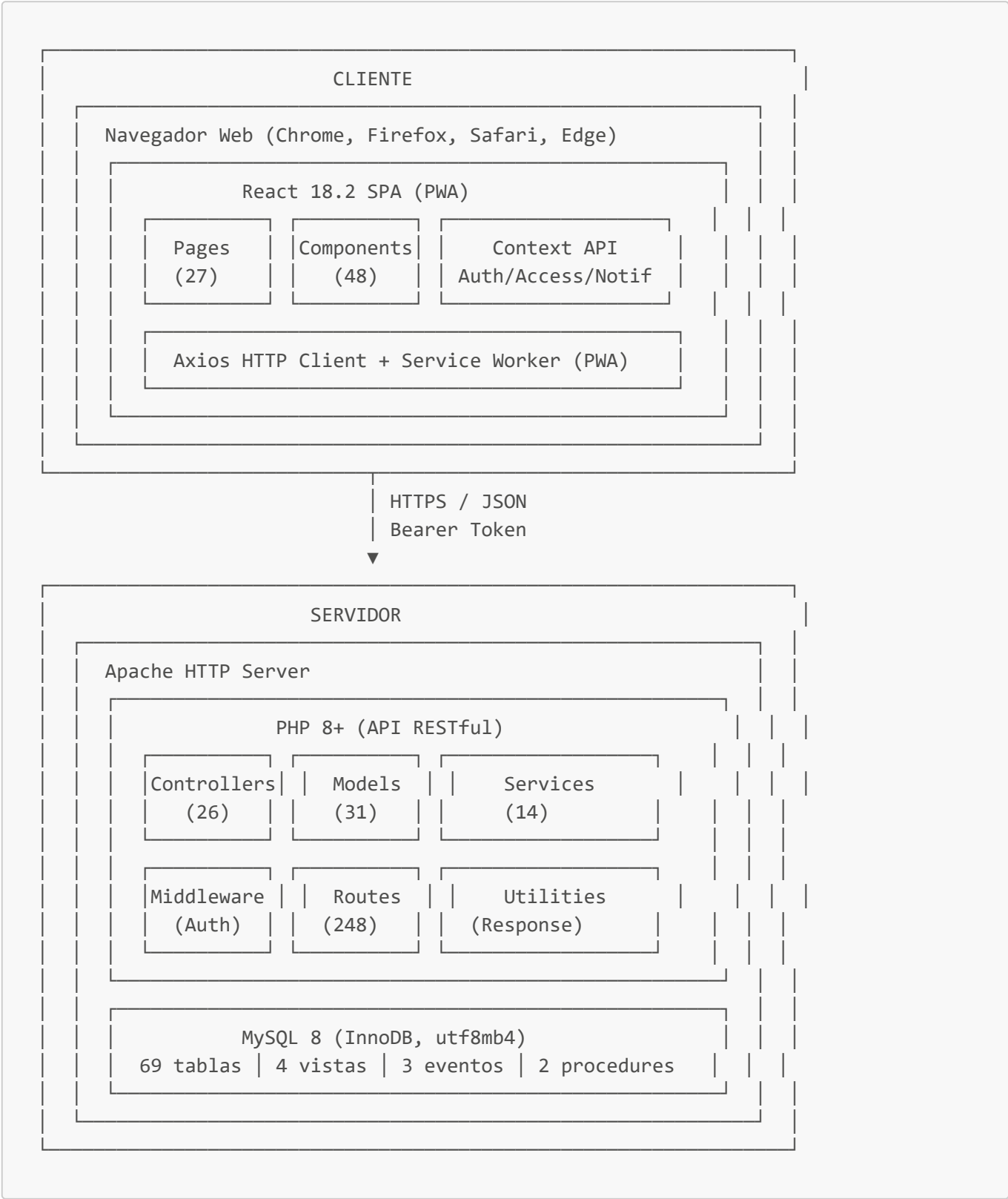
1.3 Convenciones

- Los comandos de terminal se presentan en bloques de código con prefijo \$.

- Las rutas de archivos se presentan en formato relativo al directorio raíz del proyecto (**Azaria/**).
- Los fragmentos de código incluyen comentarios explicativos cuando la lógica no es evidente.
- Las variables de entorno se presentan en mayúsculas con prefijo según el componente.

2. Arquitectura del Sistema

2.1 Diagrama de arquitectura



2.2 Patrón arquitectónico

El sistema sigue una arquitectura **cliente-servidor desacoplada** con los siguientes patrones:

- **Frontend:** Single Page Application (SPA) con React, usando Context API para estado global.
- **Backend:** API RESTful con patrón MVC manual (sin framework). Se utiliza el patrón Singleton para la conexión a base de datos.
- **Comunicación:** HTTP/JSON con autenticación Bearer Token.
- **Base de datos:** Modelo relacional con MySQL 8, acceso vía PDO con prepared statements.

2.3 Flujo de una petición

```
1. Usuario interactúa con la interfaz React
2. Componente React invoca api.get() o api.post() (Axios)
3. Axios interceptor agrega token Bearer al header Authorization
4. Petición HTTP llega a backend/public/index.php
5. index.php ejecuta CorsMiddleware y CsrftMiddleware
6. Router (api.php) encuentra la ruta coincidente
7. Si la ruta requiere auth, AuthMiddleware valida el token
8. Controller procesa la lógica de negocio
9. Controller usa DatabaseService (PDO) para consultar MySQL
10. Controller retorna Response::success($data) o Response::error($msg)
11. Axios response interceptor unwraps response.data
12. Componente React actualiza el state y re-renderiza
```

3. Requisitos del Sistema

3.1 Requisitos de hardware (servidor)

Componente	Mínimo	Recomendado
Procesador	1 core	2+ cores
RAM	1 GB	2+ GB
Almacenamiento	5 GB	20+ GB
Red	10 Mbps	100+ Mbps

3.2 Requisitos de software (servidor)

Software	Versión mínima	Versión utilizada
Sistema operativo	Linux (cualquier distro)	CentOS/Ubuntu en producción
Apache HTTP Server	2.4	2.4
PHP	8.0	8.1
MySQL	8.0	8.0
mod_rewrite	Habilitado	Habilitado

Software	Versión mínima	Versión utilizada
OpenSSL	1.1	1.1+

Extensiones PHP requeridas:

- `pdo_mysql` — Acceso a MySQL vía PDO
- `mbstring` — Manipulación de cadenas multibyte (UTF-8)
- `json` — Codificación/decodificación JSON
- `openssl` — Generación de tokens HMAC-SHA256
- `fileinfo` — Validación de tipos MIME de archivos
- `gd` o `imagick` — Procesamiento de imágenes (opcional)

3.3 Requisitos de software (desarrollo)

Software	Versión
Node.js	18 LTS
npm	9+
XAMPP	8.1+ (incluye Apache, MySQL, PHP)
Git	2.30+
Visual Studio Code	Última versión

3.4 Requisitos del cliente (navegador)

Navegador	Versión mínima
Google Chrome	90+
Mozilla Firefox	88+
Safari	14+
Microsoft Edge	90+
Samsung Internet	14+

4. Estructura del Proyecto

4.1 Árbol de directorios principal

Azaria/

├── backend/

│ ├── config/

│ │ ├── constants.php

│ │ └── database.php

│ └── public/

│ └── index.php

API RESTful PHP (44,118 líneas)

Configuración

Constantes del sistema

Conexión PDO (DatabaseService)

Punto de entrada web

Entry point, autoloader, CORS

src/	# Código fuente
Controllers/	# 26 controladores (11,279 líneas)
Middleware/	# Middleware de autenticación
Models/	# 31 modelos (3,313 líneas)
Routes/	# Definición de rutas
api.php	# 248 rutas API
Services/	# 14 servicios (2,820 líneas)
Utils/	# Utilidades
Response.php	# Respuestas JSON estandarizadas
uploads/	# Archivos subidos por usuarios
storage/	# Logs y almacenamiento temporal
logs/	
error.log	
database/	# Esquemas y migraciones
azaria_db.sql	# Esquema principal (69 tablas)
migrations/	# 21 migraciones incrementales
frontend/	# Aplicación React (33,847 líneas)
public/	# Assets estáticos
index.html	# HTML raíz
manifest.json	# Configuración PWA
service-worker.js	# Cache strategies
icon-192.png	# Icono PWA 192x192
icon-512.png	# Icono PWA 512x512
src/	# Código fuente React
components/	# 48 componentes (15,266 líneas)
accessibility/	# Panel de accesibilidad
admin/	# Componentes de administración
auth/	# Login, PIN keyboard
fisioterapia/	# Componentes de fisioterapia
layouts/	# Layouts por rol
neuropsicologia/	# Componentes de neuropsicología
nutricion/	# Componentes de nutrición
ortesis/	# Componentes de ortesis
outlook/	# Integración Outlook
shared/	# Componentes compartidos
context/	# Context API (estado global)
AuthContext.jsx	# Autenticación
AccessibilityContext.jsx	# Accesibilidad
NotificationContext.jsx	# Notificaciones
data/	# Datos estáticos
pages/	# 27 páginas (16,594 líneas)
admin/	# AdminDashboard
especialista/	# Dashboards especialista
paciente/	# Dashboard paciente
services/	# Servicios HTTP
api.js	# Instancia Axios configurada
authService.js	# Operaciones de autenticación
perfilService.js	# Operaciones de perfil
styles/	# 25 archivos CSS (21,434 líneas)
design-system.css	# Variables CSS del sistema
accessibility.css	# Estilos de accesibilidad
[módulo].css	# Un CSS por módulo
utils/	# Utilidades
constants.js	# Constantes frontend

```
├── App.jsx           # Router principal
├── index.js         # Entry point + SW registration
├── package.json     # Dependencias npm
└── Lineamientos/    # Documentos de diseño
```

4.2 Métricas del código fuente

Componente	Archivos	Líneas de código
Controllers PHP	26	11,279
Models PHP	31	3,313
Services PHP	14	2,820
Otros PHP (config, middleware, utils, routes)	~10	~26,706
Total backend PHP	~81	44,118
Páginas React (JSX)	27	16,594
Componentes React (JSX)	48	15,266
Servicios y utilidades (JS)	~8	~1,987
Total frontend JS/JSX	~83	33,847
Archivos CSS	25	21,434
SQL (esquema + migraciones)	22	~3,000
Total general	~211	~102,399

5. Base de Datos

5.1 Configuración de conexión

Entorno de desarrollo (XAMPP):

```
Host:      localhost
Puerto:   3307
Base:      vitality_db
Usuario:   root
Password:  12345
Charset:   utf8mb4
```

Entorno de producción:

```
Host:      localhost
Puerto:   3306
Base:      bd_azaria
```

```
Usuario:  azaria
Password: 12345
Charset:  utf8mb4
```

Archivo de configuración: `backend/config/database.php`

```
class DatabaseService {
    private static $instance = null;
    private $pdo;

    private function __construct() {
        $config = [
            'host'      => $_ENV['DB_HOST'] ?? 'localhost',
            'port'      => $_ENV['DB_PORT'] ?? '3307',
            'dbname'    => $_ENV['DB_NAME'] ?? 'vitalia_db',
            'username' => $_ENV['DB_USER'] ?? 'root',
            'password' => $_ENV['DB_PASS'] ?? '12345',
            'charset'   => 'utf8mb4'
        ];

        $this->pdo = new PDO(
            "mysql:host={$config['host']};port={$config['port']};
            dbname={$config['dbname']};charset={$config['charset']}",
            $config['username'],
            $config['password'],
            [
                PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
                PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
                PDO::ATTR_EMULATE_PREPARES => false
            ]
        );
    }

    public static function getInstance() {
        if (self::$instance === null) {
            self::$instance = new self();
        }
        return self::$instance;
    }
}
```

5.2 Esquema de la base de datos

El esquema se encuentra en `database/azaria_db.sql` (1,668 líneas). A continuación se presentan las tablas organizadas por grupo funcional.

5.2.1 Tablas de usuarios y autenticación

Tabla	Descripción	Campos clave
-------	-------------	--------------

Tabla	Descripción	Campos clave
usuarios	Usuarios del sistema	id, nombre, email, telefono, password_hash, rol, activo, foto_perfil
roles	Catálogo de roles	id, nombre, descripcion
sesiones	Sesiones activas	id, usuario_id, token_hash, expira_en, ip, user_agent
tokens_recuperacion	Tokens de recuperación de contraseña	id, usuario_id, codigo, expira_en, usado
log_accesos	Registro de inicios de sesión	id, usuario_id, ip, user_agent, fecha
log_auditoria	Bitácora de cambios	id, usuario_id, accion, tabla, registro_id, datos_anteriores, datos_nuevos

5.2.2 Tablas de pacientes y asignaciones

Tabla	Descripción	Campos clave
pacientes	Datos clínicos del paciente	id, usuario_id, fecha_nacimiento, tipo_amputacion, causa_amputacion, fase_actual
asignaciones_especialista	Relación paciente-especialista	id, paciente_id, especialista_id, area_medica, activa
disponibilidad_especialista	Horarios de atención	id, especialista_id, dia_semana, hora_inicio, hora_fin
fases_tratamiento	Catálogo de fases	id, nombre, descripcion, orden

5.2.3 Tablas de nutrición

Tabla	Descripción
registro_comidas	Comidas registradas por el paciente con calorías y macros
checklist_comidas	Checklist diario de alimentación
tipos_comida	Catálogo: desayuno, comida, cena, colación
cuestionarios_nutricion	Evaluaciones nutricionales
recetas	Catálogo de recetas saludables
recetas_asignadas	Recetas asignadas a pacientes
recetas_favoritas	Recetas marcadas como favoritas

5.2.4 Tablas de medicina

Tabla	Descripción
bitacora_glucosa	Mediciones de glucemia
bitacora_presion	Mediciones de presión arterial
bitacora_dolor	Registros de dolor (EVA 0-10)
medicamentos_paciente	Medicamentos activos
horarios_medimento	Horarios de toma
alertas_medicas	Alertas por valores fuera de rango

5.2.5 Tablas de fisioterapia

Tabla	Descripción
videos_ejercicios	Biblioteca de videos de ejercicios
videos_asignados	Videos asignados a pacientes
registro_videos	Registro de ejercicios completados
categorias_ejercicio	Categorías de ejercicios
niveles_ejercicio	Niveles de dificultad

5.2.6 Tablas de neuropsicología

Tabla	Descripción
cuestionarios_bienestar	Resultados de cuestionarios clínicos
emociones	Catálogo de emociones
registro_animo	Registro diario de ánimo
registro_animo_emociones	Emociones seleccionadas por registro

5.2.7 Tablas de ortesis

Tabla	Descripción
dispositivos_paciente	Dispositivos asignados
checklist_protesis	Checklist diario de uso
guias_cuidado	Guías de cuidado y mantenimiento
historial_ajustes	Registro de ajustes realizados
tipos_dispositivo	Catálogo de tipos de dispositivos

5.2.8 Tablas de admisiones

Tabla	Descripción
<code>solicitudes_admision</code>	Solicitudes de ingreso al programa
<code>documentos_admision</code>	Documentos subidos por solicitantes
<code>pagos_admision</code>	Registros de pago
<code>tokens_documentos</code>	Tokens temporales para subida de documentos
<code>documentos_oficiales_admision</code>	Documentos oficiales descargables
<code>screening_clinico</code>	Respuestas de screening de admisión

5.3 Vistas

```
-- Vista consolidada de pacientes
CREATE VIEW vista_pacientes AS
SELECT u.id, u.nombre, u.email, p.tipo_amputacion, p.fase_actual, ...
FROM usuarios u JOIN pacientes p ON u.id = p.usuario_id;

-- Vista de especialistas
CREATE VIEW vista_especialistas AS
SELECT u.id, u.nombre, u.email, am.nombre AS area, COUNT(ae.id) AS num_pacientes
FROM usuarios u LEFT JOIN areas_medicas am ... LEFT JOIN asignaciones_especialista
ae ...;

-- Vista de citas pendientes
CREATE VIEW vista_citas_pendientes AS
SELECT c.*, u_pac.nombre AS paciente, u_esp.nombre AS especialista
FROM citas c JOIN usuarios u_pac ... JOIN usuarios u_esp ... WHERE c.estado IN
('pendiente', 'confirmada');

-- Vista de alertas pendientes
CREATE VIEW vista_alertas_pendientes AS
SELECT am.*, u.nombre AS paciente FROM alertas_medicas am JOIN usuarios u ...
WHERE am.estado = 'pendiente';
```

5.4 Eventos programados

```
-- Elimina mensajes de chat con más de 90 días
CREATE EVENT eliminar_mensajes_expirados
ON SCHEDULE EVERY 1 DAY DO
DELETE FROM mensajes_chat WHERE fecha < DATE_SUB(NOW(), INTERVAL 90 DAY);

-- Limpia tokens de recuperación expirados
CREATE EVENT limpiar_tokens_expirados
ON SCHEDULE EVERY 1 HOUR DO
DELETE FROM tokens_recuperacion WHERE expira_en < NOW();
```

```
-- Cierra sesiones inactivas
CREATE EVENT limpiar_sesiones_expiradas
ON SCHEDULE EVERY 1 HOUR DO
DELETE FROM sesiones WHERE expira_en < NOW();
```

5.5 Procedimientos almacenados

```
-- Calcula porcentaje de adherencia a medicamentos
DELIMITER //
CREATE PROCEDURE sp_adherencia_medicamentos(IN p_paciente_id INT, IN p_dias INT)
BEGIN
    -- Calcula: (tomas registradas / tomas esperadas) * 100
    -- Retorna: porcentaje, tomas_esperadas, tomas_registradas
END //

-- Genera resumen estadístico de bitácoras
CREATE PROCEDURE sp_resumen_bitacoras(IN p_paciente_id INT, IN p_dias INT)
BEGIN
    -- Retorna: promedios, máximos, mínimos de glucosa, presión y dolor
END //
DELIMITER ;
```

5.6 Migraciones

Las migraciones se ubican en `database/migrations/` y se nombran con la convención `AAAAMMDD_NNNNNN_descripcion.sql`.

#	Archivo	Descripción
1	20260120_000001_add_macros_to_registro_comidas.sql	Agrega columnas de macronutrientes
2	20260131_000001_planes_nutricionales.sql	Tablas de planes nutricionales
3	20260207_000001_neuropsicologia_act.sql	Tablas de actividades ACT
4	20260207_000002_expediente_archivos.sql	Tabla de archivos del expediente
5	20260209_000001_configuracion_usuario.sql	Preferencias de usuario
6	20260219_000001_mediciones_antropometricas.sql	Tabla antropometría
7	20260220_000001_generador_plan_nutricional.sql	Generador de planes PDF
8	20260222_000001_fisioterapia_especialista.sql	Fisioterapia para especialistas
9	20260301_000001_admisiones.sql	Sistema de admisiones completo
10	20260301_000002_datos_prueba_admisiones.sql	Datos de prueba
11	20260301_000003_usuario_mariana.sql	Usuario administrativo
12	20260304_000001_bitacora_hba1c.sql	Bitácora de HbA1c

#	Archivo	Descripción
13	20260304_000002_datos_ortesis.sql	Datos iniciales de ortesis
14	20260309_000001_comunidad_imagen.sql	Imágenes en comunidad
15	20260309_000002_cuestionarios_personalizados.sql	Cuestionarios personalizados
16	20260309_000003_cuestionarios_interactivos.sql	Motor de cuestionarios
17	20260310_000001_ortesis_mediciones_protocolo.sql	Mediciones de muñón
18	20260310_000002_act_asignaciones.sql	Asignaciones ACT
19	20260310_000003_act_asignaciones_contenido.sql	Contenido ACT
20	20260313_000001_screening_clinico.sql	Screening de admisión
21	evaluacion_neuropsicologica.sql	Evaluaciones neuropsicológicas

Para ejecutar una migración:

```
$ C:/xampp/mysql/bin/mysql.exe -u root -p12345 -P 3307 vitalia_db <
database/migrations/archivo.sql
```

6. Backend - API RESTful

6.1 Punto de entrada (index.php)

El archivo `backend/public/index.php` es el punto de entrada de todas las peticiones al backend. Sus responsabilidades son:

1. **Autoloading:** Intenta cargar Composer; si no existe, usa un autoloader PSR-4 manual que mapea el namespace `App\` al directorio `src/`.
2. **Variables de entorno:** Carga `.env` si existe.
3. **CORS:** Ejecuta `CorsMiddleware::handle()` para permitir peticiones cross-origin.
4. **CSRF:** Ejecuta `CsrfMiddleware::handle()` para peticiones POST/PUT/DELETE.
5. **Archivos estáticos:** Sirve archivos de `/uploads/` con headers de caché.
6. **Enrutamiento:** Normaliza la URI (elimina prefijo `/~azaria/api` o `/api`) y carga las rutas de `api.php`.
7. **404:** Retorna JSON `{"success": false, "message": "Ruta no encontrada"}` si no hay coincidencia.

6.2 Sistema de enrutamiento

El router se implementa en `backend/src/Routes/api.php` mediante la función global `route()`:

```
function route($method, $pattern, $callback, $options = []) {
    // $method: 'GET', 'POST', 'PUT', 'DELETE'
    // $pattern: regex, ej: '/api/nutricion/historial/(\d+)'
    // $callback: función que recibe parámetros capturados
```

```
// $options: ['auth'] = requiere token, [] = público, ['rate:auth'] = rate
limit
}
```

Ejemplo de definición de rutas:

```
// Ruta pública (sin autenticación)
route('POST', '/api/admisiones/solicitud', function() {
    $controller = new AdmisionesController();
    $controller->crearSolicitud();
}, []);

// Ruta protegida (requiere token)
route('GET', '/api/nutricion/historial/(\d+)', function($pacienteId) {
    $controller = new NutricionController();
    $controller->getHistorialComidas($pacienteId);
}, ['auth']);

// Ruta con role de administrador
route('GET', '/api/admin/metricas', function() {
    $controller = new AdminController();
    $controller->getMetricas();
}, ['auth']);
```

6.3 Controladores

Los controladores siguen un patrón consistente:

```
namespace App\Controllers;

use App\Services\DatabaseService;
use App\Middleware\AuthMiddleware;
use App\Utils\Response;

class MiController {
    private $db;

    public function __construct() {
        $this->db = DatabaseService::getInstance();
    }

    public function miMetodo($id) {
        $user = AuthMiddleware::getCurrentUser();

        try {
            $result = $this->db->query(
                "SELECT * FROM mi_tabla WHERE id = ?",
                [$id]
            )->fetchAll();
        } catch (Exception $e) {
            // Manejar error
        }
    }
}
```

```
        return Response::success($result);
    } catch (\Exception $e) {
        error_log('Error: ' . $e->getMessage());
        return Response::error('Error al procesar', 500);
    }
}
```

Lista de controladores

Controlador	Líneas	Métodos	Módulo
PlanNutricionalController	2,630	12	Planes nutricionales
AdmisionesController	946	14	Admisiones
OrtesisController	832	15	Ortesis
NeuropsicologiaController	716	14	Neuropsicología
CitasController	515	9	Citas
ExpedienteController	477	7	Expediente clínico
MedicinaController	448	13	Medicina
OutlookCalendarController	423	10	Outlook
NutricionController	411	10	Nutrición
FisioterapiaController	404	12	Fisioterapia
MensajesController	392	5	Mensajes/chat
GeneradorPlanController	358	3	Generador planes
AdminController	349	9	Administración
EspecialistaController	330	6	Especialistas
RecetaController	320	7	Recetas
ComunidadController	297	9	Comunidad
ConfiguracionController	254	4	Configuración
AuthController	251	8	Autenticación
BlogController	170	8	Blog
AntropometriaController	164	5	Antropometría
RecordatoriosController	126	4	Recordatorios
FAQController	122	1	FAQs
PerfilController	99	2	Perfil

Controlador	Líneas	Métodos	Módulo
DashboardController	97	1	Dashboard
ChatController	84	2	Chat
FaseController	64	5	Fases

6.4 Modelos

Los modelos proporcionan acceso estructurado a las tablas de la base de datos. Cada modelo encapsula las consultas SQL relacionadas con una entidad.

```
namespace App\Models;

use App\Services\DatabaseService;

class MiModelo {
    private $db;
    private $table = 'mi_tabla';

    public function __construct() {
        $this->db = DatabaseService::getInstance();
    }

    public function getById($id) {
        return $this->db->query(
            "SELECT * FROM {$this->table} WHERE id = ?", [$id]
        )->fetch();
    }

    public function create($data) {
        $this->db->query(
            "INSERT INTO {$this->table} (campo1, campo2) VALUES (?, ?)",
            [$data['campo1'], $data['campo2']]
        );
        return $this->db->lastInsertId();
    }
}
```

6.5 Servicios

Servicio	Líneas	Descripción
EmailService	770	Envío de correos SMTP con plantillas HTML
AlimentosDatabase	445	Base de datos de alimentos mexicanos
MicrosoftGraphService	411	Integración con Microsoft Graph API
AuthService	391	Validación de credenciales y sesiones

Servicio	Líneas	Descripción
SessionService	261	Gestión de tokens HMAC-SHA256
EncryptionService	192	Cifrado y descifrado de datos sensibles
PINService	106	Gestión de PIN numérico
FileUploadService	75	Subida y validación de archivos
DatabaseService	77	Singleton de conexión PDO a MySQL
GoogleCalendarService	25	Integración Google Calendar (stub)
ModerationService	20	Moderación de contenido (stub)
NotificationService	16	Notificaciones (stub)
RecordatorioService	19	Lógica de recordatorios (stub)
ChartService	12	Generación de datos para gráficas (stub)

6.6 Clase Response

Todas las respuestas del API se estandarizan mediante **Response**:

```
class Response {
    public static function success($data = null, $message = 'OK', $code = 200) {
        http_response_code($code);
        header('Content-Type: application/json');
        echo json_encode([
            'success' => true,
            'data' => $data,
            'message' => $message
        ]);
        exit;
    }

    public static function error($message = 'Error', $code = 400) {
        http_response_code($code);
        header('Content-Type: application/json');
        echo json_encode([
            'success' => false,
            'message' => $message
        ]);
        exit;
    }
}
```

7. Frontend - Aplicación React

7.1 Dependencias

package.json:

```
{
  "name": "azaria-frontend",
  "homepage": "/~azaria/",
  "dependencies": {
    "react": "18.2.0",
    "react-dom": "18.2.0",
    "react-router-dom": "6.20.0",
    "axios": "1.6.2",
    "chart.js": "4.4.0",
    "react-chartjs-2": "5.2.0",
    "date-fns": "3.0.0",
    "react-icons": "5.5.0",
    "lucide-react": "0.574.0",
    "jspdf": "4.2.0",
    "html2canvas": "1.4.1",
    "pdfjs-dist": "3.11.174"
  },
  "devDependencies": {
    "react-scripts": "5.0.1"
  }
}
```

7.2 Servicio API (api.js)

```
import axios from 'axios';

const api = axios.create({
  baseURL: process.env.REACT_APP_API_URL || 'http://localhost:8000/api',
  headers: {
    'Content-Type': 'application/json',
    'X-Requested-With': 'XMLHttpRequest'
  }
});

// Interceptor de petición: agrega token Bearer
api.interceptors.request.use(config => {
  const token = localStorage.getItem('token');
  if (token) {
    config.headers.Authorization = `Bearer ${token}`;
  }
  return config;
});

// Interceptor de respuesta: unwrap data + manejo de 401
api.interceptors.response.use(
  response => response.data, // Retorna directamente response.data
  error => {
    if (error.response?.status === 401) {

```

```

        localStorage.removeItem('token');
        window.location.href = '/login';
    }
    return Promise.reject(error.response?.data || { message: 'Error de red'
});
}
);

export default api;

```

Nota importante: El interceptor de respuesta hace `unwrap` automático de `response.data`. Esto significa que al llamar `await api.get('/endpoint')`, el resultado ya es el objeto `{success, data, message}` del backend, NO el objeto response de Axios. No se debe hacer `.data.data`.

7.3 Context API

AuthContext

```

const AuthContext = createContext();

export const AuthProvider = ({ children }) => {
    const [user, setUser] = useState(null);
    const [loading, setLoading] = useState(true);

    const login = async (credentials) => { /* ... */ };
    const logout = () => { /* ... */ };
    const setupPIN = async (pin) => { /* ... */ };

    return (
        <AuthContext.Provider value={{ user, loading, login, logout, setupPIN }}>
            {children}
        </AuthContext.Provider>
    );
};

export const useAuth = () => useContext(AuthContext);

```

AccessibilityContext

Proporciona 13 configuraciones de accesibilidad persistidas en `localStorage`:

```

const defaultSettings = {
    fontSize: 18, // px (14-24)
    highContrast: false,
    readingMode: false,
    textSpacing: false,
    largeCursor: false,
    voiceEnabled: false,

```

```

    reducedMotion: false,
    colorBlindMode: 'none', // 'none', 'protanopia', 'deuteranopia', 'tritanopia'
    underlineLinks: false,
    focusHighlight: false,
    brightness: 100,        // % (50-150)
    saturation: 100,        // % (0-200)
    largeFormText: false
  };

```

7.4 Enrutamiento (App.jsx)

```

<BrowserRouter basename={process.env.REACT_APP_BASENAME || '/~azaria'}>
  <AuthProvider>
    <AccessibilityProvider>
      <VoiceProvider>
        <NotificationProvider>
          <Routes>
            {/* Rutas públicas */}
            <Route path="/login" element={<Login />} />
            <Route path="/solicitud" element={<Solicitud />} />
            <Route path="/admisiones/estatus" element=
{<EstatusAdmision />} />
            <Route path="/admisiones/documentos/:token" element=
{<SubirDocumentos />} />
            <Route path="/expediente/compartido/:token" element=
{<ExpedienteCompartido />} />
            <Route path="/recuperar-password" element=
{<RecuperarPassword />} />

            {/* Ruta raíz - redirección por rol */}
            <Route path="/" element={<DashboardRedirect />} />

            {/* Rutas de administrador */}
            <Route path="/admin" element={<AdminRoute><AdminDashboard
/></AdminRoute>} />

            {/* Rutas de especialista */}
            <Route path="/especialista" element={
              <EspecialistaRoute><EspecialistaDashboard />
            } />
            <Route
path="/especialista/pacientes/:pacienteId/expediente" element={
              <EspecialistaRoute><ExpedientePaciente />
            } />
            </EspecialistaRoute>

            {/* Rutas de módulos (protegidas + ModuleLayout) */}
            <Route path="/nutricion" element={
              <ProtectedRoute><ModuleLayout><Nutricion />
            } />
            </ModuleLayout></ProtectedRoute>
          </Routes>
        </NotificationProvider>
      </VoiceProvider>
    </AccessibilityProvider>
  </AuthProvider>
</BrowserRouter>

```

```

    } />
    { /* ... demás módulos ... */}

    { /* 404 */}
    <Route path="*" element={<NotFound />} />
  </Routes>
</NotificationProvider>
</VoiceProvider>
</AccessibilityProvider>
</AuthProvider>
</BrowserRouter>

```

7.5 Sistema de diseño CSS

Archivo: `frontend/src/styles/design-system.css` (563 líneas)

Colores por módulo

```

:root {
  --color-primary: #0097A7;
  --color-nutricion: #4CAF50;
  --color-fisioterapia: #FF9800;
  --color-medicina: #F44336;
  --color-neuropsicologia: #9C27B0;
  --color-ortesis: #00BCD4;
  --color-citas: #009688;
  --color-config: #607D8B;
  --color-blog: #3F51B5;
  --color-comunidad: #E91E63;
  --color-recordatorios: #FFC107;
  --color-dashboard: #2196F3;
}

```

Tema oscuro (por defecto)

```

:root {
  --background-primary: #0D1117;
  --surface-primary: #161B22;
  --surface-secondary: #21262D;
  --surface-tertiary: #30363D;
  --border-color: #30363D;
  --text-primary: #E6EDF3;
  --text-secondary: #8B949E;
  --text-muted: #6E7681;
}

```

Espaciado y bordes

```
:root {  
  --spacing-xs: 4px;  
  --spacing-sm: 8px;  
  --spacing-md: 16px;  
  --spacing-lg: 24px;  
  --spacing-xl: 32px;  
  --radius-sm: 8px;  
  --radius-md: 12px;  
  --radius-lg: 16px;  
  --radius-xl: 24px;  
}
```

Controles táctiles (accesibilidad)

```
:root {  
  --touch-target-min: 48px;  
  --touch-target-comfortable: 56px;  
  --font-size-base: 18px; /* Grande por defecto para adultos mayores */  
}
```

7.6 Patrón de página típica

```
import React, { useState, useEffect } from 'react';  
import { useAuth } from '../context/AuthContext';  
import api from '../services/api';  
import '../styles/MiModulo.css';  
  
const MiModulo = () => {  
  const { user } = useAuth();  
  const [data, setData] = useState(null);  
  const [loading, setLoading] = useState(true);  
  
  useEffect(() => {  
    cargarDatos();  
  }, []);  
  
  const cargarDatos = async () => {  
    try {  
      const res = await api.get(`/mi-modulo/${user.id}`);  
      setData(res?.data || null);  
    } catch (error) {  
      console.error('Error:', error);  
    } finally {  
      setLoading(false);  
    }  
  }  
};
```

```

    if (loading) return <div className="loading-spinner">Cargando...</div>;

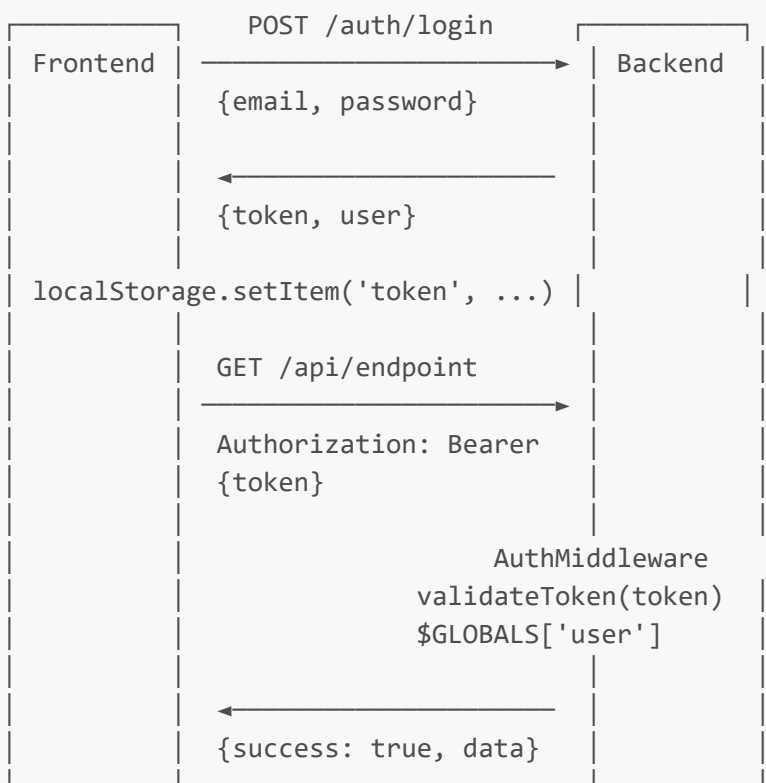
    return (
      <div className="mi-modulo-container">
        <h1>Mi Módulo</h1>
        { /* Contenido */ }
      </div>
    );
  };
};

export default MiModulo;

```

8. Sistema de Autenticación

8.1 Flujo de autenticación



8.2 Generación de tokens

```

// SessionService.php
public function createSession($userId, $rememberMe = false) {
    $token = bin2hex(random_bytes(32)); // 64 caracteres hex
    $tokenHash = hash_hmac('sha256', $token, $this->secretKey);
    $expiresAt = $rememberMe
        ? date('Y-m-d H:i:s', strtotime('+30 days'))
        : date('Y-m-d H:i:s', strtotime('+24 hours'));
}

```

```

    $this->db->query(
        "INSERT INTO sesiones (usuario_id, token_hash, expira_en, ip, user_agent)
        VALUES (?, ?, ?, ?, ?)",
        [$userId, $tokenHash, $expiresAt, $_SERVER['REMOTE_ADDR'],
$_SERVER['HTTP_USER_AGENT']]
    );

    return $token; // Se retorna el token original, no el hash
}

```

8.3 Validación de tokens

```

// SessionService.php
public function validateToken($token) {
    $tokenHash = hash_hmac('sha256', $token, $this->secretKey);

    $session = $this->db->query(
        "SELECT s.*, u.* FROM sesiones s
        JOIN usuarios u ON s.usuario_id = u.id
        WHERE s.token_hash = ? AND s.expira_en > NOW()",
        [$tokenHash]
    )->fetch();

    return $session ?: null;
}

```

8.4 Middleware de autenticación

```

// AuthMiddleware.php
class AuthMiddleware {
    public static function handle() {
        $user = SessionService::getCurrentUser();
        if (!$user) {
            return Response::error('No autorizado', 401);
        }
        $GLOBALS['current_user'] = $user;
    }

    public static function getCurrentUser() {
        return $GLOBALS['current_user'] ?? null;
    }
}

```

8.5 Roles y permisos

Rol	Valor	Acceso
-----	-------	--------

Rol	Valor	Acceso
paciente	'paciente'	Módulos de salud, citas, chat, expediente propio
especialista	'especialista'	Dashboard especialista, expedientes de pacientes asignados
administrador	'administrador'	Panel admin, CRUD usuarios, admisiones, métricas, FAQs, blog

Frontend - Guards de ruta:

```
// ProtectedRoute - cualquier usuario autenticado
const ProtectedRoute = ({ children }) => {
  const { user, loading } = useAuth();
  if (loading) return <Loading />;
  if (!user) return <Navigate to="/login" />;
  return children;
};

// RoleBasedRoute - rol específico
const AdminRoute = ({ children }) => {
  const { user } = useAuth();
  if (user?.rol !== 'administrador') return <Navigate to="/" />;
  return children;
};
```

9. Sistema de Accesibilidad

9.1 Panel de accesibilidad

El componente `AccessibilityPanel.jsx` (420 líneas) proporciona un panel flotante accesible desde cualquier página del sistema.

9.2 Opciones configurables

#	Opción	Tipo	Rango	Default
1	Tamaño de fuente	Slider	14px - 24px	18px
2	Alto contraste	Toggle	on/off	off
3	Modo de lectura	Toggle	on/off	off
4	Espaciado de texto	Toggle	on/off	off
5	Cursor aumentado	Toggle	on/off	off
6	Narrador de voz	Toggle	on/off	off
7	Reducción de animaciones	Toggle	on/off	off
8	Modo daltónico	Select	none/protanopia/deuteranopia/tritanopia	none

#	Opción	Tipo	Rango	Default
9	Subrayado de enlaces	Toggle	on/off	off
10	Foco visible aumentado	Toggle	on/off	off
11	Brillo	Slider	50% - 150%	100%
12	Saturación	Slider	0% - 200%	100%
13	Texto grande en formularios	Toggle	on/off	off

9.3 Narrador de voz (VoiceHelper)

El componente `VoiceHelper.jsx` (551 líneas) implementa un asistente de voz que:

- Lee en voz alta los títulos de las secciones al navegar
- Narra las instrucciones de los formularios
- Proporciona retroalimentación auditiva de acciones (guardado, error)
- Usa la Web Speech API nativa del navegador
- Configurable en idioma (es-MX por defecto) y velocidad

9.4 Persistencia

Las configuraciones se almacenan en `localStorage` con la clave `vitalia-accessibility` en formato JSON, y se aplican al cargar la aplicación mediante atributos `data-*` en el elemento raíz del DOM:

```
document.documentElement.setAttribute('data-font-scale', settings.fontScale);
document.documentElement.setAttribute('data-contrast', settings.contrast);
document.documentElement.setAttribute('data-reduced-motion',
settings.reducedMotion);
```

10. Progressive Web App (PWA)

10.1 Manifest.json

```
{
  "name": "Azaria - Tu Compañero de Salud",
  "short_name": "Azaria",
  "display": "standalone",
  "orientation": "portrait",
  "lang": "es-MX",
  "theme_color": "#0D1117",
  "background_color": "#0D1117",
  "categories": ["health", "medical", "lifestyle"],
  "icons": [
    { "src": "icon-192.png", "sizes": "192x192", "type": "image/png" },
    { "src": "icon-512.png", "sizes": "512x512", "type": "image/png" }
  ],
}
```

```
"shortcuts": [
  { "name": "Mis Recordatorios", "url": "./recordatorios" },
  { "name": "Registrar Comida", "url": "./nutricion" },
  { "name": "Mis Ejercicios", "url": "./fisioterapia" }
]
```

10.2 Service Worker

El archivo `service-worker.js` implementa estrategias de caché:

- **Cache-first:** Para assets estáticos (JS, CSS, imágenes). Si está en caché, se sirve desde ahí; si no, se descarga y se almacena.
- **Network-first:** Para peticiones API. Intenta la red primero; si falla, busca en caché (permite funcionamiento offline parcial).
- **Stale-while-revalidate:** Para recursos que cambian poco (fuentes, iconos). Sirve la versión en caché mientras actualiza en segundo plano.

11. Catálogo de Rutas API

11.1 Rutas públicas (sin autenticación)

Método	Ruta	Descripción
POST	<code>/api/auth/login</code>	Iniciar sesión
POST	<code>/api/auth/forgot-password</code>	Solicitar código de recuperación
POST	<code>/api/auth/verify-code</code>	Verificar código de recuperación
POST	<code>/api/auth/reset-password</code>	Restablecer contraseña
POST	<code>/api/auth/register</code>	Registrar usuario
POST	<code>/api/auth/microsoft/callback</code>	Callback OAuth Microsoft
POST	<code>/api/admisiones/solicitud</code>	Crear solicitud de admisión
POST	<code>/api/admisiones/documentos/{token}</code>	Subir documentos por token
GET	<code>/api/admisiones/documentos/{token}</code>	Ver documentos subidos
GET	<code>/api/admisiones/documentos-oficiales</code>	Listar documentos oficiales
GET	<code>/api/admisiones/documentos-oficiales/{id}/descargar</code>	Descargar documento oficial
POST	<code>/api/admisiones/estatus</code>	Consultar estatus de solicitud
GET	<code>/api/expediente/compartido/{token}</code>	Ver expediente compartido

11.2 Rutas de autenticación y perfil

Método	Ruta	Descripción
POST	/api/auth/logout	Cerrar sesión
POST	/api/auth/setup-pin	Configurar PIN numérico
GET	/api/auth/check-session	Verificar sesión activa
GET	/api/auth/devices	Listar dispositivos de confianza
DELETE	/api/auth/devices/{id}	Eliminar dispositivo
POST	/api/auth/logout-all-devices	Cerrar todas las sesiones
PUT	/api/auth/cambiar-password	Cambiar contraseña
PUT	/api/auth/cambiar-pin	Cambiar PIN
GET	/api/perfil	Obtener perfil del usuario
PUT	/api/perfil	Actualizar perfil

11.3 Rutas de nutrición

Método	Ruta	Descripción
GET	/api/nutricion/historial/{pacienteId}	Historial de comidas
POST	/api/nutricion/comidas	Registrar comida
GET	/api/nutricion/checklist/{pacienteId}/{fecha}	Checklist diario
POST	/api/nutricion/checklist	Actualizar checklist
GET	/api/nutricion/resumen/{pacienteId}/{fecha}	Resumen del día
POST	/api/nutricion/agua	Registrar consumo de agua
GET	/api/nutricion/agua/{pacienteId}/{fecha}	Registro de agua
POST	/api/nutricion/alimento	Registrar alimento
GET	/api/nutricion/alimentos/buscar	Buscar alimentos
GET	/api/nutricion/recetas	Listar recetas
GET	/api/nutricion/recetas/catalogo	Catálogo de recetas
GET	/api/nutricion/recetas/catalogo/{id}	Detalle de receta
POST	/api/nutricion/recetas/catalogo	Crear receta
PUT	/api/nutricion/recetas/catalogo/{id}	Actualizar receta
DELETE	/api/nutricion/recetas/catalogo/{id}	Eliminar receta
GET	/api/nutricion/recetas/por-tipo	Recetas por tipo

Método	Ruta	Descripción
POST	/api/nutricion/antropometria/{pacienteId}	Registrar medición
GET	/api/nutricion/antropometria/{pacienteId}	Obtener mediciones
GET	/api/nutricion/antropometria/{pacienteId}/ultima	Última medición
GET	/api/nutricion/antropometria/{pacienteId}/peso	Evolución de peso
DELETE	/api/nutricion/antropometria/medicion/{id}	Eliminar medición

11.4 Rutas de planes nutricionales

Método	Ruta	Descripción
GET	/api/nutricion/planes/especialista/{id}	Planes del especialista
POST	/api/nutricion/planes/upload/{id}	Subir plan PDF
GET	/api/nutricion/planes/{id}	Obtener plan
PUT	/api/nutricion/planes/{id}	Actualizar contenido
DELETE	/api/nutricion/planes/{id}	Eliminar plan
POST	/api/nutricion/planes/{id}/asignar	Asignar plan a paciente
GET	/api/nutricion/plan-paciente/{pacienteId}	Plan del paciente
POST	/api/nutricion/plan-paciente/{pacienteId}/seguimiento	Registrar seguimiento
POST	/api/nutricion/planes/{id}/recetas	Agregar recetas a plan
DELETE	/api/nutricion/planes/{id}/recetas/{comidaId}	Quitar receta
POST	/api/nutricion/planes/{id}/imagenes	Subir imagen a plan
DELETE	/api/nutricion/planes/{id}/imagenes	Eliminar imagen
POST	/api/nutricion/planes/generar	Generar plan automático
GET	/api/nutricion/planes/generado/{id}	Ver plan generado
PUT	/api/nutricion/planes/generado/{id}	Actualizar plan generado

11.5 Rutas de medicina

Método	Ruta	Descripción
GET	/api/medicina/glucosa/{pacienteId}	Historial de glucosa
POST	/api/medicina/glucosa	Registrar glucosa
GET	/api/medicina/presion/{pacienteId}	Historial de presión
POST	/api/medicina/presion	Registrar presión

Método	Ruta	Descripción
GET	/api/medicina/dolor/{pacienteId}	Historial de dolor
POST	/api/medicina/dolor	Registrar dolor
GET	/api/medicina/resumen/{pacienteId}	Resumen de salud
GET	/api/medicina/hba1c/{pacienteId}	Historial HbA1c
POST	/api/medicina/hba1c	Registrar HbA1c
GET	/api/medicina/medicamentos/{pacienteId}	Medicamentos activos
POST	/api/medicina/medicamentos	Crear medicamento
PUT	/api/medicina/medicamentos/{id}	Actualizar medicamento
DELETE	/api/medicina/medicamentos/{id}	Eliminar medicamento

11.6 Rutas de fisioterapia

Método	Ruta	Descripción
GET	/api/fisioterapia/videos	Listar videos
GET	/api/fisioterapia/videos/{id}	Detalle de video
GET	/api/fisioterapia/rutina/{pacienteId}	Videos asignados
GET	/api/fisioterapia/progreso/{pacienteId}	Historial de progreso
GET	/api/fisioterapia/guias	Guías de ejercicios
GET	/api/fisioterapia/checklist/{pacienteId}	Checklist diario
POST	/api/fisioterapia/checklist	Guardar checklist
POST	/api/fisioterapia/ejercicio/completar	Marcar ejercicio completado
GET	/api/fisioterapia/evaluaciones/{pacienteId}	Evaluaciones del paciente
GET	/api/fisioterapia/evaluacion/{id}	Detalle de evaluación
POST	/api/fisioterapia/evaluaciones	Crear evaluación
PUT	/api/fisioterapia/evaluaciones/{id}	Actualizar evaluación
DELETE	/api/fisioterapia/evaluaciones/{id}	Eliminar evaluación
GET	/api/fisioterapia/planes/paciente/{pacienteId}	Planes del paciente
GET	/api/fisioterapia/planes/{id}	Detalle de plan
POST	/api/fisioterapia/planes	Crear plan
PUT	/api/fisioterapia/planes/{id}	Actualizar plan
PUT	/api/fisioterapia/planes/{id}/estado	Cambiar estado del plan

Método	Ruta	Descripción
GET	<code>/api/fisioterapia/stats/paciente/{pacienteId}</code>	Estadísticas

11.7 Rutas de neuropsicología

Método	Ruta	Descripción
GET	<code>/api/neuropsicologia/estados-animos/{pacienteId}</code>	Historial de ánimo
POST	<code>/api/neuropsicologia/estados-animos</code>	Registrar estado de ánimo
GET	<code>/api/neuropsicologia/ejercicios</code>	Ejercicios disponibles
GET	<code>/api/neuropsicologia/cuestionarios/{pacienteId}</code>	Cuestionarios del paciente
POST	<code>/api/neuropsicologia/cuestionarios</code>	Guardar cuestionario
POST	<code>/api/neuropsicologia/cuestionarios/resultados</code>	Guardar resultado
GET	<code>/api/neuropsicologia/cuestionarios/historial/{pacienteId}</code>	Historial
GET	<code>/api/neuropsicologia/cuestionarios-personalizados</code>	Cuestionarios custom
POST	<code>/api/neuropsicologia/cuestionarios-personalizados</code>	Crear cuestionario custom
DELETE	<code>/api/neuropsicologia/cuestionarios-personalizados/{id}</code>	Eliminar
GET	<code>/api/neuropsicologia/mis-cuestionarios/{pacienteId}</code>	Cuestionarios asignados
POST	<code>/api/neuropsicologia/mis-cuestionarios/responder</code>	Responder cuestionario
GET	<code>/api/neuropsicologia/mis-cuestionarios/historial/{pacienteId}</code>	Historial respuestas
POST	<code>/api/neuropsicologia/act/sesion</code>	Guardar sesión ACT
GET	<code>/api/neuropsicologia/act/historial/{pacienteId}</code>	Historial ACT
GET	<code>/api/neuropsicologia/act/asignaciones/{pacienteId}</code>	Asignaciones ACT
POST	<code>/api/neuropsicologia/act/asignar</code>	Asignar herramienta ACT
DELETE	<code>/api/neuropsicologia/act/asignaciones/{id}</code>	Eliminar asignación
PUT	<code>/api/neuropsicologia/act/asignaciones/{id}/completar</code>	Completar asignación

Método	Ruta	Descripción
PUT	/api/neuropsicologia/act/asignaciones/{id}/estado	Cambiar estado
GET	/api/neuropsicologia/evaluacion/{pacienteId}	Evaluación actual
GET	/api/neuropsicologia/evaluacion/historial/{pacienteId}	Historial evaluaciones
POST	/api/neuropsicologia/evaluacion	Guardar evaluación

11.8 Rutas de ortesis

Método	Ruta	Descripción
GET	/api/protesis/educativo	Contenido educativo
GET	/api/protesis/niveles-k	Niveles K
GET	/api/protesis/tipos	Tipos de prótesis
GET	/api/protesis/guias	Guías de cuidado
GET	/api/protesis/faqs	FAQs de prótesis
GET	/api/protesis/videos	Videos educativos
GET	/api/ortesis/dispositivo/{pacienteId}	Dispositivo del paciente
PUT	/api/ortesis/dispositivo/{pacienteId}/nivel-k	Actualizar nivel K
GET	/api/ortesis/checklist/{pacienteId}/{fecha}	Checklist diario
GET	/api/ortesis/problemas/{pacienteId}	Problemas reportados
POST	/api/ortesis/problemas	Reportar problema
GET	/api/ortesis/ajustes/{pacienteId}	Historial de ajustes
POST	/api/ortesis/ajustes/{pacienteId}	Crear ajuste
PUT	/api/ortesis/problemas/{id}/resolver	Resolver problema
GET	/api/ortesis/mediciones-munon/{pacienteId}	Mediciones de muñón
POST	/api/ortesis/mediciones-munon/{pacienteId}	Crear medición
GET	/api/ortesis/protocolo-uso/{pacienteId}	Registros de uso
POST	/api/ortesis/protocolo-uso/{pacienteId}	Crear registro de uso
GET	/api/ortesis/protocolo-uso/{pacienteId}/config	Config protocolo
POST	/api/ortesis/protocolo-uso/{pacienteId}/config	Guardar config

11.9 Rutas de citas

Método	Ruta	Descripción
--------	------	-------------

Método	Ruta	Descripción
GET	/api/citas	Mis citas
POST	/api/citas	Agendar cita
GET	/api/citas/tipos	Tipos de cita
GET	/api/citas/horarios/{especialistaId}/{fecha}	Horarios disponibles
GET	/api/citas/especialista/{especialistaId}	Citas del especialista
POST	/api/citas/especialista	Crear cita (especialista)
PUT	/api/citas/{id}/estado	Cambiar estado
PUT	/api/citas/{id}/notas	Agregar notas
PUT	/api/citas/{id}/cancelar	Cancelar cita

11.10 Rutas de comunicación

Método	Ruta	Descripción
GET	/api/chat/conversaciones	Listar conversaciones
POST	/api/chat/mensajes	Enviar mensaje
GET	/api/mensajes/no-leidos/{userId}	Mensajes no leídos
GET	/api/mensajes/conversaciones/{userId}	Conversaciones del usuario
GET	/api/mensajes/conversacion/{id}/{userId}	Mensajes de conversación
POST	/api/mensajes/enviar	Enviar mensaje
POST	/api/mensajes/iniciar/{userId}/{otroId}	Iniciar conversación

11.11 Rutas de administración

Método	Ruta	Descripción
GET	/api/admin/metricas	Métricas del sistema
GET	/api/admin/usuarios	Listar usuarios
POST	/api/admin/usuarios	Crear usuario
PUT	/api/admin/usuarios/{id}	Actualizar usuario
DELETE	/api/admin/usuarios/{id}	Eliminar usuario
PUT	/api/admin/usuarios/{id}/toggle	Activar/desactivar usuario
GET	/api/admin/admisiones	Listar solicitudes
GET	/api/admin/admisiones/{id}	Detalle de solicitud

Método	Ruta	Descripción
PUT	/api/admin/admisiones/{id}/estado	Cambiar estado
POST	/api/admin/admisiones/{id}/pagado	Marcar como pagado
POST	/api/admin/admisiones/{id}/token-documentos	Generar token docs
PUT	/api/admin/admisiones/{id}/preconsulta	Programar preconsulta
POST	/api/admin/admisiones/{id}/admitir	Admitir paciente
PUT	/api/admin/admisiones/{id}/rechazar	Rechazar solicitud
GET	/api/admin/admisiones/reportes/semestre	Reporte semestral

11.12 Rutas de expediente

Método	Ruta	Descripción
GET	/api/expediente/resumen/{pacienteId}	Resumen del expediente
POST	/api/expediente/archivos	Subir archivo
GET	/api/expediente/archivos/{pacienteId}	Listar archivos
DELETE	/api/expediente/archivos/{id}	Eliminar archivo
GET	/api/expediente/archivos/{id}/descargar	Descargar archivo
POST	/api/expediente/compartir/{pacienteId}	Generar enlace compartido

11.13 Rutas de Outlook

Método	Ruta	Descripción
GET	/api/outlook/status	Estado de conexión
GET	/api/outlook/connected	Verificar conexión
GET	/api/outlook/auth	Iniciar OAuth
DELETE	/api/outlook/disconnect	Desconectar
POST	/api/outlook/sync/{citaId}	Sincronizar cita
GET	/api/outlook/events	Obtener eventos
POST	/api/outlook/availability	Consultar disponibilidad
PUT	/api/outlook/sync/{citaId}	Actualizar evento
DELETE	/api/outlook/sync/{citaId}	Eliminar evento

12. Configuración del Entorno de Desarrollo

12.1 Prerequisitos

1. Instalar **XAMPP 8.1+** desde <https://www.apachefriends.org/>
2. Instalar **Node.js 18 LTS** desde <https://nodejs.org/>
3. Instalar **Git** desde <https://git-scm.com/>
4. Instalar **Visual Studio Code** desde <https://code.visualstudio.com/>

12.2 Configuración de la base de datos

```
# 1. Iniciar XAMPP (Apache y MySQL en puerto 3307)

# 2. Crear la base de datos
$ C:/xampp/mysql/bin/mysql.exe -u root -p12345 -P 3307 -e "CREATE DATABASE
vitalia_db CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci"

# 3. Importar el esquema principal
$ C:/xampp/mysql/bin/mysql.exe -u root -p12345 -P 3307 vitalia_db <
database/azaria_db.sql

# 4. Ejecutar migraciones (en orden cronológico)
$ for file in database/migrations/20*.sql; do
    C:/xampp/mysql/bin/mysql.exe -u root -p12345 -P 3307 vitalia_db < "$file"
    echo "Migración aplicada: $file"
done
```

12.3 Configuración del backend

```
# 1. Verificar que PHP esté en el PATH
$ php -v
# PHP 8.1.x

# 2. Iniciar servidor de desarrollo
$ cd backend
$ php -S localhost:8000 -t public

# El servidor queda escuchando en http://localhost:8000/api
```

Variables de entorno opcionales (archivo `backend/.env`):

```
DB_HOST=localhost
DB_PORT=3307
DB_NAME=vitalia_db
DB_USER=root
DB_PASS=12345

MAIL_HOST=smtp.gmail.com
MAIL_PORT=587
MAIL_USERNAME=tu_correo@gmail.com
MAIL_PASSWORD=tu_app_password
```

```
MAIL_FROM=noreply@azaria.app
MAIL_FROM_NAME=Azaria - UIOP

MICROSOFT_CLIENT_ID=tu_client_id
MICROSOFT_CLIENT_SECRET=tu_client_secret
MICROSOFT_REDIRECT_URI=http://localhost:3000/~azaria/citas
```

12.4 Configuración del frontend

```
# 1. Instalar dependencias
$ cd frontend
$ npm install

# 2. Crear archivo .env (opcional para desarrollo)
$ echo "REACT_APP_API_URL=http://localhost:8000/api" > .env

# 3. Iniciar servidor de desarrollo
$ npm start

# La aplicación queda disponible en http://localhost:3000/~azaria/
```

12.5 Verificación

1. Abrir <http://localhost:3000/~azaria/login> en el navegador
2. Iniciar sesión con credenciales de prueba
3. Verificar que el dashboard carga correctamente
4. Probar un endpoint API directamente: <http://localhost:8000/api/auth/check-session>

13. Configuración del Entorno de Producción

13.1 Datos del servidor

```
Host:          dtai.uteq.edu.mx
Usuario SSH:   azaria
Password SSH:  Azhar1aa_2026*
Home:         /home/aazaria
Public HTML:   /home/aazaria/public_html
Base URL:      https://dtai.uteq.edu.mx/~azaria/
```

13.2 Estructura en producción

```
/home/aazaria/
├─ public_html/
│   ├── index.html      # Frontend build (React)
│   └─ static/          # JS/CSS bundles
```

```
|— manifest.json      # PWA config
|— service-worker.js  # Service Worker
|— icon-192.png
|— icon-512.png
|— .htaccess          # Rewrite rules para SPA + API
|— api/               # Backend PHP
|   |— public/
|   |   |— index.php
|   |— src/
|   |   |— Controllers/
|   |   |— Models/
|   |   |— Routes/
|   |   |— Services/
|   |   |— Middleware/
|   |   |— Utils/
|   |— config/
|   |— uploads/
|   |— storage/
|   |   |— logs/
```

13.3 Configuración de Apache (.htaccess)

```
RewriteEngine On
```

```
# API requests → backend
```

```
RewriteRule ^api/(.*)$ api/public/index.php [QSA,L]
```

```
# Archivos estáticos existentes → servir directamente
```

```
RewriteCond %{REQUEST_FILENAME} -f
```

```
RewriteRule ^ - [L]
```

```
# Todo lo demás → index.html (SPA routing)
```

```
RewriteCond %{REQUEST_FILENAME} !-f
```

```
RewriteCond %{REQUEST_FILENAME} !-d
```

```
RewriteRule ^ index.html [L]
```

13.4 Base de datos en producción

```
Host:      localhost
Puerto:   3306
Base:      bd_azaria
Usuario:   azaria
Password:  12345
```

14. Procedimientos de Despliegue

14.1 Despliegue automático (script Python)

Se utiliza un script Python con la librería `paramiko` para transferencias SFTP:

```
import paramiko
import os

# Configuración
HOST = 'dtai.uteq.edu.mx'
USER = 'azaria'
PASS = 'Azhar1aa_2026*'
REMOTE_BASE = '/home/aazaria/public_html'

def deploy():
    ssh = paramiko.SSHClient()
    ssh.set_missing_host_key_policy(paramiko.AutoAddPolicy())
    ssh.connect(HOST, username=USER, password=PASS)
    sftp = ssh.open_sftp()

    # 1. Subir backend
    for root, dirs, files in os.walk('backend/src'):
        for file in files:
            local = os.path.join(root, file)
            remote = os.path.join(REMOTE_BASE, 'api',
                                   root.replace('backend/', ''), file)
            sftp.put(local, remote)

    # 2. Subir frontend build
    for root, dirs, files in os.walk('frontend/build'):
        for file in files:
            local = os.path.join(root, file)
            remote = os.path.join(REMOTE_BASE,
                                   root.replace('frontend/build/', ''), file)
            sftp.put(local, remote)

    sftp.close()
    ssh.close()

deploy()
```

14.2 Despliegue manual paso a paso

Frontend:

```
# 1. Generar build de producción
$ cd frontend
$ npm run build

# 2. Subir archivos vía SFTP al servidor
$ sftp azaria@dtai.uteq.edu.mx
```

```
sftp> cd public_html
sftp> put -r build/*
```

Backend:

```
# Subir archivos PHP modificados
$ sftp azaria@dtai.uteq.edu.mx
sftp> cd public_html/api
sftp> put -r src/
sftp> put -r config/
```

Migraciones de base de datos:

```
# 1. Subir archivo de migración al servidor
$ scp database/migrations/archivo.sql azaria@dtai.uteq.edu.mx:/tmp/

# 2. Ejecutar migración vía SSH
$ ssh azaria@dtai.uteq.edu.mx
$ mysql -u azaria -p12345 bd_azaria < /tmp/archivo.sql
```

14.3 Verificación post-despliegue

1. Acceder a <https://dtai.uteq.edu.mx/~azaria/> y verificar que cargue el login
2. Iniciar sesión y navegar por los módulos principales
3. Verificar la consola del navegador (F12) en busca de errores 404 o 500
4. Probar el formulario público de solicitud: <https://dtai.uteq.edu.mx/~azaria/solicitud>
5. Verificar los logs del servidor: `tail -f ~/public_html/api/storage/logs/error.log`

15. Mantenimiento y Administración

15.1 Logs

Ubicación: `/home/aazaria/public_html/api/storage/logs/error.log`

```
# Ver últimas 50 líneas de error
$ ssh azaria@dtai.uteq.edu.mx "tail -50 public_html/api/storage/logs/error.log"

# Buscar errores específicos
$ ssh azaria@dtai.uteq.edu.mx "grep '500' public_html/api/storage/logs/error.log"
```

15.2 Respaldo de base de datos

```
# Respaldo completo
$ ssh azaria@dtai.uteq.edu.mx "mysqldump -u azaria -p12345 bd_azaria >
~/backup_$(date +%Y%m%d).sql"

# Descargar respaldo
$ scp azaria@dtai.uteq.edu.mx:~/backup_*.sql ./backups/
```

15.3 Limpieza de archivos temporales

```
# Limpiar sesiones expiradas (normalmente lo hace el evento MySQL)
$ mysql -u azaria -p12345 bd_azaria -e "DELETE FROM sesiones WHERE expira_en <
NOW()"

# Limpiar tokens de documentos expirados
$ mysql -u azaria -p12345 bd_azaria -e "DELETE FROM tokens_documentos WHERE
expira_en < NOW()"
```

15.4 Monitoreo de rendimiento

Tamaño de la base de datos:

```
SELECT table_name AS 'Tabla',
       ROUND(data_length / 1024 / 1024, 2) AS 'Datos (MB)',
       ROUND(index_length / 1024 / 1024, 2) AS 'Índices (MB)',
       table_rows AS 'Registros'
FROM information_schema.tables
WHERE table_schema = 'bd_azaria'
ORDER BY data_length DESC;
```

Sesiones activas:

```
SELECT COUNT(*) AS sesiones_activas,
       COUNT(CASE WHEN expira_en > NOW() THEN 1 END) AS vigentes,
       COUNT(CASE WHEN expira_en <= NOW() THEN 1 END) AS expiradas
FROM sesiones;
```

15.5 Actualización de correos SMTP

Si se necesita cambiar las credenciales de correo, editar el archivo `backend/.env` en producción:

```
MAIL_HOST=smtp.gmail.com
MAIL_PORT=587
MAIL_USERNAME=nuevo_correo@gmail.com
MAIL_PASSWORD=nueva_contraseña_de_aplicacion
```

Para Gmail, la contraseña debe ser una "Contraseña de aplicación" generada en <https://myaccount.google.com/apppasswords>.

16. Solución de Problemas

16.1 Error 500 en endpoints API

Causa más común: Error de SQL o referencia a tabla/columna inexistente.

Diagnóstico:

1. Revisar `storage/logs/error.log` en el servidor
2. Buscar el mensaje de error específico
3. Verificar que la tabla/columna exista en la base de datos

Solución: Ejecutar la migración faltante o corregir la consulta SQL.

16.2 Error 401 "No autorizado"

Causas posibles:

- Token expirado (24h normal, 30d con "recordarme")
- Token no se envía en el header
- Sesión eliminada del servidor

Solución:

1. Verificar que `localStorage.getItem('token')` retorna un valor
2. Verificar que el token no ha expirado en la tabla `sesiones`
3. Si el problema persiste, cerrar sesión e iniciar de nuevo

16.3 Página en blanco después del despliegue

Causa: El `basename` del router no coincide con la ruta en el servidor.

Verificar:

- `package.json` → `"homepage": "/~azaria/"`
- `.htaccess` → Reglas de rewrite configuradas
- `frontend/src/App.jsx` → `<BrowserRouter basename={...}>`

16.4 Correos no se envían

Verificar:

1. Variables `MAIL_*` configuradas en `.env`
2. Puerto 587 no bloqueado por firewall
3. Contraseña de aplicación válida (no la contraseña normal de Gmail)
4. Revisar logs: `grep 'SMTP' storage/logs/error.log`

16.5 Error CORS

Causa: Petición desde un dominio no permitido.

Solución: Verificar la configuración en `CorsMiddleware.php` y asegurar que los headers `Access-Control-Allow-Origin` incluyen el dominio del frontend.

16.6 Archivos subidos no se visualizan

Verificar:

1. Permisos de la carpeta `uploads/` (chmod 755)
2. Que el archivo existe en la ruta correcta
3. Que el `.htaccess` o `index.php` permite servir archivos de `uploads/`
4. Tipos MIME soportados: jpg, jpeg, png, gif, bmp, pdf, webp, doc, docx

17. Glosario

Término	Definición
ACT	Terapia de Aceptación y Compromiso (Acceptance and Commitment Therapy)
API	Interfaz de Programación de Aplicaciones (Application Programming Interface)
Bearer Token	Esquema de autenticación HTTP donde el token se envía en el header Authorization
CORS	Cross-Origin Resource Sharing - mecanismo de seguridad del navegador
CRUD	Create, Read, Update, Delete - operaciones básicas de datos
CSRF	Cross-Site Request Forgery - tipo de ataque web
EVA	Escala Visual Analógica - escala de dolor de 0 a 10
GAD-7	Escala de 7 ítems para evaluar trastorno de ansiedad generalizada
HbA1c	Hemoglobina glicosilada - indicador de control glucémico
HMAC-SHA256	Código de autenticación de mensajes basado en hash
IMC	Índice de Masa Corporal
MVC	Modelo-Vista-Controlador - patrón de arquitectura de software
Nivel K	Clasificación funcional de movilidad para usuarios de prótesis (K0-K4)
PDO	PHP Data Objects - interfaz de acceso a bases de datos
PHQ-9	Cuestionario de Salud del Paciente de 9 ítems para evaluar depresión
PWA	Progressive Web Application - aplicación web con capacidades nativas
REST	Representational State Transfer - estilo arquitectónico para APIs
SPA	Single Page Application - aplicación de una sola página

Término	Definición
SMTP	Simple Mail Transfer Protocol - protocolo de envío de correo
UIOP	Unidad de Investigación en Órtesis y Prótesis
WCAG	Web Content Accessibility Guidelines - directrices de accesibilidad web

Manual Técnico generado para la Plataforma Azaria v1.0 Abril 2026 - C2DEVELOPERS